

Actividad 3: Interfaz Gráfica de Usuario
Programación Orientada a Objetos

Ejercicios 8.2 y 8.3 – Ejercicios de programación orientada a objetos con Java y UML
Leonardo Bermón Angarita

Facultad de Ingeniería

Universidad Nacional de Colombia
Docente: Walter Hugo Arboleda Mazo

Ximena Muñoz Lara
Octubre de 2026

Contenido

Ejercicio 8.2 – Notas.....3
Interfaz de usuario:.....5
Diagrama de Clase:.....6
Diagrama de Objetos:.....6
Casos de uso:.....7
URL en GitHub:.....7
Ejercicio 8.3 – Figuras geométricas.....8
Interfaz de usuario:.....13
Diagrama de Clase:.....14
Diagrama de Objetos:.....16
Casos de uso:.....17
URL en GitHub:.....17
URL del repositorio en GitHub:.....17

Ejercicio 8.2 – Notas

Enunciado: Se requiere desarrollar un programa con interfaz gráfica de usuario que genere una ventana donde se solicite el ingreso de cinco notas obtenidas por un estudiante. El programa debe calcular y mostrar en la parte inferior de la ventana el promedio de las notas ingresadas, la desviación estándar, la mayor nota obtenida y la menor nota obtenida.

Parte propuesta:

1. *Generar mensajes de alerta cuando se ingresen datos que no sean numéricos.*
2. *Es obligatorio ingresar las cinco notas: se genera un mensaje de alerta cuando una nota no ha sido ingresada.*

Archivo 1: notas.py

```
import math


class Notas:
    def __init__(self):
        # arreglo con las 5 notas
        self.lista_notas = [0, 0, 0, 0, 0]

    def calcular_promedio(self):
        suma = 0
        for nota in self.lista_notas:
            suma += nota
        return suma / len(self.lista_notas)

    def calcular_desviacion(self):
        prom = self.calcular_promedio()
        suma = 0
        for nota in self.lista_notas:
            suma += (nota - prom) ** 2
        return math.sqrt(suma / len(self.lista_notas))

    def calcular_menor(self):
        menor = self.lista_notas[0]
        for nota in self.lista_notas:
            if nota < menor:
                menor = nota
        return menor

    def calcular_mayor(self):
        mayor = self.lista_notas[0]
        for nota in self.lista_notas:
            if nota > mayor:
                mayor = nota
        return mayor
```

Archivo 2: ventana_notas.py

```
import tkinter as tk
from tkinter import messagebox
from notas import Notas


class VentanaNotas(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Notas")
        self.resizable(False, False)

        # campos de texto para las 5 notas
        self.campos = []
        for i in range(5):
            etiqueta = tk.Label(self, text=f"Nota {i + 1}:", anchor="w")
            etiqueta.place(x=20, y=20 + 30 * i, width=80, height=23)
            campo = tk.Entry(self)
            campo.place(x=105, y=20 + 30 * i, width=135, height=23)
            self.campos.append(campo)

        # botones
        self.boton_calcular = tk.Button(self, text="Calcular", command=self.calcular)
        self.boton_calcular.place(x=20, y=170, width=100, height=23)
        self.boton_limpiar = tk.Button(self, text="Limpiar", command=self.limpiar)
        self.boton_limpiar.place(x=125, y=170, width=80, height=23)

        # etiquetas donde se muestran los resultados
        self.promedio = tk.Label(self, text="Promedio = ", anchor="w")
        self.promedio.place(x=20, y=210, width=230, height=23)
        self.desviacion = tk.Label(self, text="Desviación estándar = ", anchor="w")
        self.desviacion.place(x=20, y=240, width=230, height=23)
        self.mayor = tk.Label(self, text="Nota mayor = ", anchor="w")
        self.mayor.place(x=20, y=270, width=230, height=23)
        self.menor = tk.Label(self, text="Nota menor = ", anchor="w")
        self.menor.place(x=20, y=300, width=230, height=23)

        # ventana centrada en la pantalla
        x = (self.winfo_screenwidth() - 280) // 2
        y = (self.winfo_screenheight() - 350) // 2
        self.geometry(f"280x350+{x}+{y}")

    # evento del boton Calcular
    def calcular(self):
        notas = Notas()

        for i in range(5):
            texto = self.campos[i].get().strip().replace(",", ".")

            # parte propuesta: las 5 notas son obligatorias
```

```
            if texto == "":
                messagebox.showwarning("Alerta", f"Falta ingresar la nota {i + 1}.")
                return

            # parte propuesta: solo se aceptan numeros
            try:
                notas.lista_notas[i] = float(texto)
            except ValueError:
                messagebox.showerror("Error", f"La nota {i + 1} no es un número válido.")
                return

        self.promedio.config(text=f"Promedio = {notas.calcular_promedio():.2f}")
        self.desviacion.config(text=f"Desviación estándar = {notas.calcular_desviacion():.2f}")
        self.mayor.config(text=f"Nota mayor = {notas.calcular_mayor()}")
        self.menor.config(text=f"Nota menor = {notas.calcular_menor()}")

    # evento del boton Limpiar
    def limpiar(self):
        for campo in self.campos:
            campo.delete(0, tk.END)
        self.promedio.config(text="Promedio = ")
        self.desviacion.config(text="Desviación estándar = ")
        self.mayor.config(text="Nota mayor = ")
        self.menor.config(text="Nota menor = ")
```

Archivo 3: principal_notas.py

```
from ventana_notas import VentanaNotas


class Principal:
    def main():
        ventana = VentanaNotas()
        ventana.mainloop()


if __name__ == "__main__":
    Principal.main()
```

Interfaz de usuario:

Captura de pantalla pendiente *a) Ventana con los resultados*	*Captura de pantalla pendiente* *b) Mensaje de alerta (falta una nota)*

Datos de ejemplo: notas 4.5, 3.0, 5.0, 3.8 y 4.2 → promedio 4.10, desviación estándar 0.68, nota mayor 5.0 y nota menor 3.0.

Diagrama de Clase:

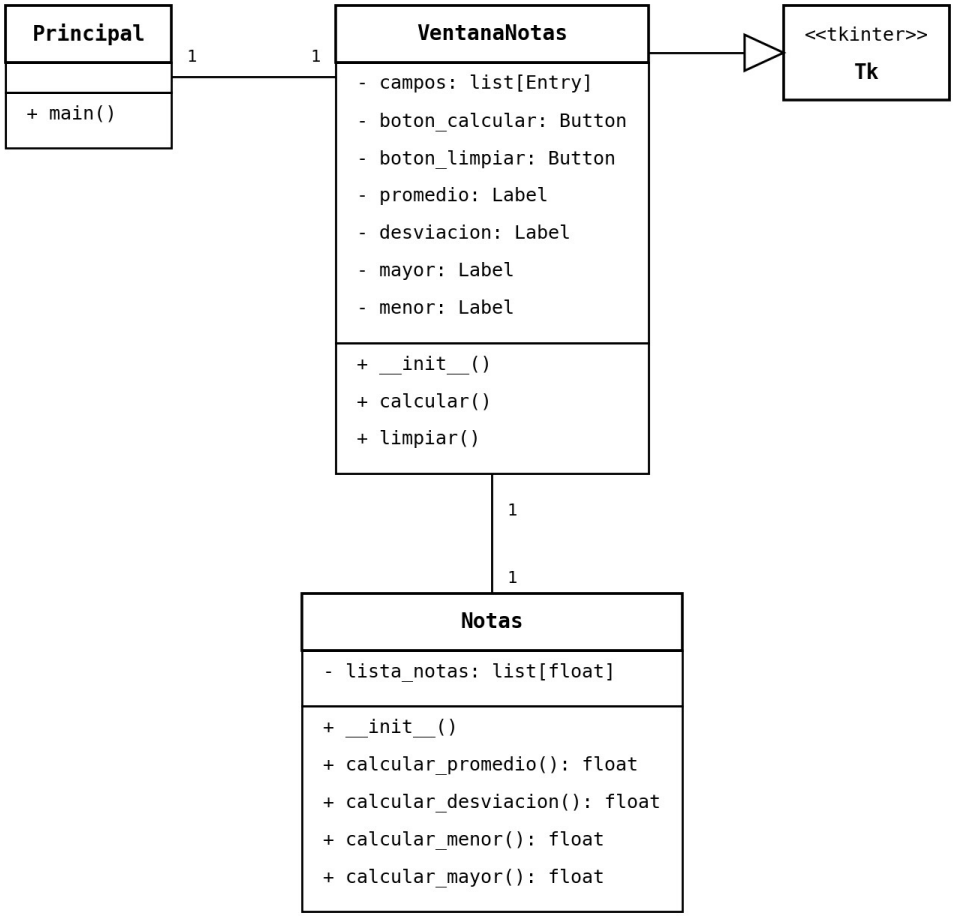
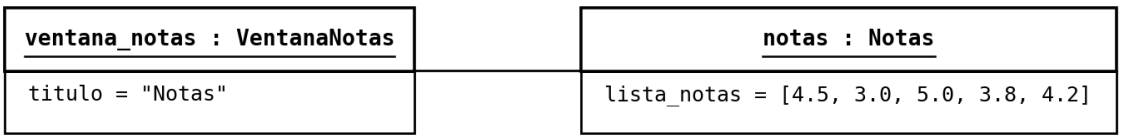
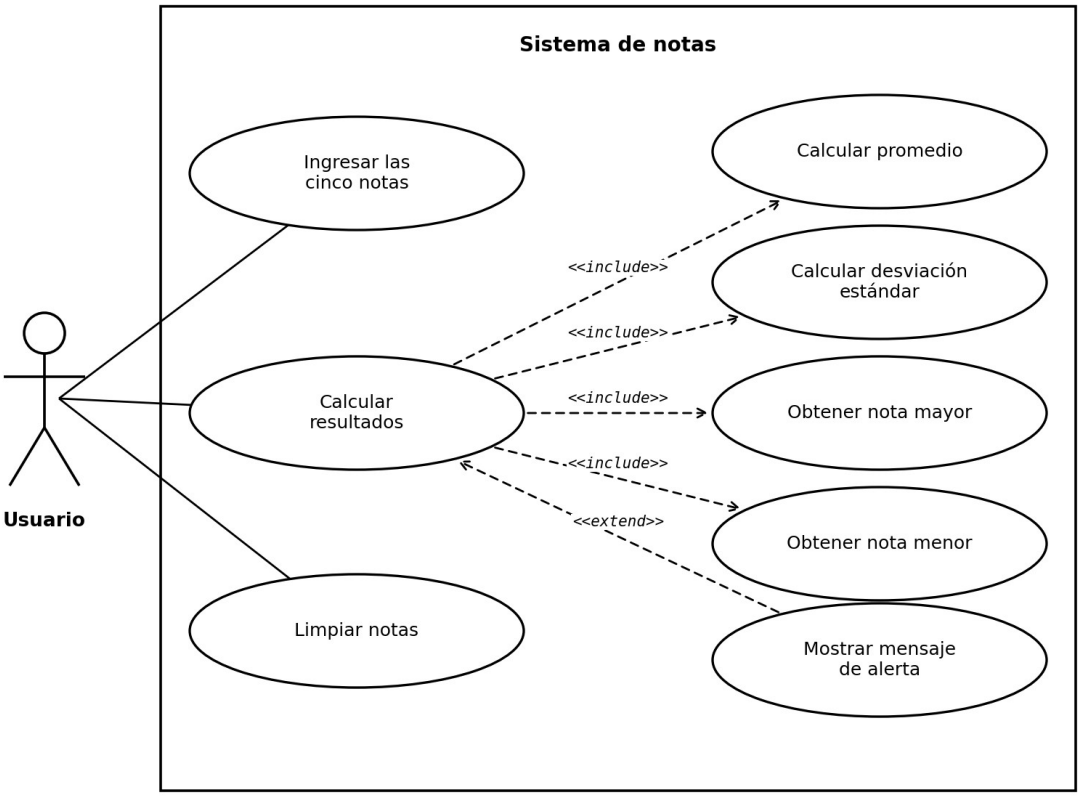


Diagrama de Objetos:



Casos de uso:



Caso de uso	Descripción
Ingresar las cinco notas	El usuario escribe en los cinco campos de texto las notas del estudiante.
Calcular resultados	Al pulsar el botón Calcular el sistema lee las notas y muestra los resultados en la parte inferior de la ventana.
Calcular promedio	Suma las cinco notas y las divide entre cinco.
Calcular desviación estándar	Calcula la raíz cuadrada del promedio de los cuadrados de las diferencias respecto al promedio.
Obtener nota mayor / menor	Recorre las notas y determina cuál es la mayor y cuál es la menor.
Limpiar notas	Al pulsar el botón Limpiar se borran los campos de texto y los resultados mostrados.
Mostrar mensaje de alerta	Parte propuesta: se muestra cuando falta alguna de las cinco notas o cuando un dato no es numérico.

URL en GitHub:

[Pegar aquí el URL de GitHub: ejercicio 8.2]

Ejercicio 8.3 – Figuras geométricas

Enunciado: Se requiere desarrollar un programa con interfaz gráfica de usuario que permita calcular el volumen y la superficie de varias figuras geométricas: el cilindro (radio y altura), la esfera (radio) y la pirámide (base, altura y apotema), con todas las medidas en centímetros. Una vez ingresados los datos, el programa calcula el volumen y la superficie de cada figura. Para desarrollar el programa se debe crear una jerarquía de clases para las diferentes figuras geométricas requeridas.

Parte propuesta:

1. *Agregar el cálculo del volumen y la superficie de otras figuras geométricas como el cubo y el prisma (se usó un prisma rectangular con largo, ancho y altura).*
2. *Agregar a cada figura geométrica su imagen correspondiente.*

Archivo 1: figura_geometrica.py

```
class FiguraGeometrica:
    def __init__(self):
        self.volumen = 0
        self.superficie = 0

    def set_volumen(self, volumen):
        self.volumen = volumen

    def set_superficie(self, superficie):
        self.superficie = superficie

    def get_volumen(self):
        return self.volumen

    def get_superficie(self):
        return self.superficie
```

Archivo 2: cilindro.py

```
import math
from figura_geometrica import FiguraGeometrica


class Cilindro(FiguraGeometrica):
    def __init__(self, radio, altura):
        super().__init__()
        self.radio = radio
        self.altura = altura
        # al crear el cilindro se calculan volumen y superficie
        self.set_volumen(self.calcular_volumen())
        self.set_superficie(self.calcular_superficie())

    def calcular_volumen(self):
        return math.pi * self.altura * self.radio ** 2
```

```
    def calcular_superficie(self):
        area_lado = 2 * math.pi * self.radio * self.altura
        area_bases = 2 * math.pi * self.radio ** 2
        return area_lado + area_bases
```

Archivo 3: esfera.py

```
import math
from figura_geometrica import FiguraGeometrica


class Esfera(FiguraGeometrica):
    def __init__(self, radio):
        super().__init__()
        self.radio = radio
        self.set_volumen(self.calcular_volumen())
        self.set_superficie(self.calcular_superficie())

    def calcular_volumen(self):
        return 4 / 3 * math.pi * self.radio ** 3

    def calcular_superficie(self):
        return 4 * math.pi * self.radio ** 2
```

Archivo 4: piramide.py

```
from figura_geometrica import FiguraGeometrica


class Piramide(FiguraGeometrica):
    def __init__(self, base, altura, apotema):
        super().__init__()
        self.base = base
        self.altura = altura
        self.apotema = apotema
        self.set_volumen(self.calcular_volumen())
        self.set_superficie(self.calcular_superficie())

    def calcular_volumen(self):
        return self.base ** 2 * self.altura / 3

    def calcular_superficie(self):
        # area de la base + area de las 4 caras laterales
        area_base = self.base ** 2
        area_lados = 2 * self.base * self.apotema
        return area_base + area_lados
```

Archivo 5: cubo.py

```
from figura_geometrica import FiguraGeometrica
```

```
class Cubo(FiguraGeometrica):
    def __init__(self, lado):
        super().__init__()
        self.lado = lado
        self.set_volumen(self.calcular_volumen())
        self.set_superficie(self.calcular_superficie())

    def calcular_volumen(self):
        return self.lado ** 3

    def calcular_superficie(self):
        return 6 * self.lado ** 2
```

Archivo 6: prisma.py

```
from figura_geometrica import FiguraGeometrica


# prisma rectangular
class Prisma(FiguraGeometrica):
    def __init__(self, largo, ancho, altura):
        super().__init__()
        self.largo = largo
        self.ancho = ancho
        self.altura = altura
        self.set_volumen(self.calcular_volumen())
        self.set_superficie(self.calcular_superficie())

    def calcular_volumen(self):
        return self.largo * self.ancho * self.altura

    def calcular_superficie(self):
        return 2 * (self.largo * self.ancho + self.largo * self.altura + self.ancho * self.altura)
```

Archivo 7: ventana_figura.py

```
import os
import tkinter as tk
from tkinter import messagebox


class VentanaFigura(tk.Toplevel):
    def __init__(self, padre, titulo, campos, clase_figura, imagen):
        super().__init__(padre)
        self.title(titulo)
        self.resizable(False, False)
        self.clase_figura = clase_figura
```

```
        # un campo de texto por cada dato que pide la figura
        self.campos = []
        for i in range(len(campos)):
            etiqueta = tk.Label(self, text=campos[i], anchor="w")
            etiqueta.place(x=20, y=20 + 30 * i, width=110, height=23)
            campo = tk.Entry(self)
            campo.place(x=130, y=20 + 30 * i, width=100, height=23)
            self.campos.append(campo)

        y = 20 + 30 * len(campos)
        self.boton_calcular = tk.Button(self, text="Calcular", command=self.calcular)
        self.boton_calcular.place(x=130, y=y, width=100, height=23)

        self.volumen = tk.Label(self, text="Volumen (cm3):", anchor="w")
        self.volumen.place(x=20, y=y + 40, width=240, height=23)
        self.superficie = tk.Label(self, text="Superficie (cm2):", anchor="w")
        self.superficie.place(x=20, y=y + 70, width=240, height=23)

        # parte propuesta: imagen de la figura
        try:
            ruta = os.path.join(os.path.dirname(os.path.abspath(__file__)), imagen)
            self.foto = tk.PhotoImage(file=ruta)
            tk.Label(self, image=self.foto).place(x=270, y=20)
        except tk.TclError:
            pass  # si no esta la imagen la ventana funciona igual

        # ventana centrada en la pantalla
        ancho = 410
        alto = y + 110
        x_ventana = (self.winfo_screenwidth() - ancho) // 2
        y_ventana = (self.winfo_screenheight() - alto) // 2
        self.geometry(f"{ancho}x{alto}+{x_ventana}+{y_ventana}")

    # evento del boton Calcular
    def calcular(self):
        valores = []
        try:
            for campo in self.campos:
                valores.append(float(campo.get().replace(",", ".")))
        except ValueError:
            messagebox.showerror("Error", "Campo nulo o error en formato de número", parent=self)
            return

        for valor in valores:
            if valor <= 0:
                messagebox.showerror("Error", "Los valores deben ser mayores que cero", parent=self)
                return

        # se crea la figura con los datos ingresados
        figura = self.clase_figura(*valores)
```

```
        self.volumen.config(text=f"Volumen (cm3): {figura.get_volumen():.2f}")
        self.superficie.config(text=f"Superficie (cm2): {figura.get_superficie():.2f}")
```

Archivo 8: ventana_figuras.py

```
import tkinter as tk
from ventana_figura import VentanaFigura
from cilindro import Cilindro
from esfera import Esfera
from piramide import Piramide
from cubo import Cubo
from prisma import Prisma


class VentanaFiguras(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Figuras")
        self.resizable(False, False)

        tk.Label(self, text="Seleccione una figura geométrica:").place(x=20, y=15)

        # un boton por cada figura
        boton_cilindro = tk.Button(self, text="Cilindro", command=self.abrir_cilindro)
        boton_cilindro.place(x=20, y=50, width=80, height=23)
        boton_esfera = tk.Button(self, text="Esfera", command=self.abrir_esfera)
        boton_esfera.place(x=110, y=50, width=80, height=23)
        boton_piramide = tk.Button(self, text="Pirámide", command=self.abrir_piramide)
        boton_piramide.place(x=200, y=50, width=80, height=23)
        boton_cubo = tk.Button(self, text="Cubo", command=self.abrir_cubo)
        boton_cubo.place(x=290, y=50, width=80, height=23)
        boton_prisma = tk.Button(self, text="Prisma", command=self.abrir_prisma)
        boton_prisma.place(x=380, y=50, width=80, height=23)

        # ventana centrada en la pantalla
        x = (self.winfo_screenwidth() - 480) // 2
        y = (self.winfo_screenheight() - 110) // 2
        self.geometry(f"480x110+{x}+{y}")

    # eventos de los botones: cada uno abre la ventana de su figura
    def abrir_cilindro(self):
        VentanaFigura(self, "Cilindro", ["Radio (cms):", "Altura (cms):"], Cilindro, "cilindro.png")

    def abrir_esfera(self):
        VentanaFigura(self, "Esfera", ["Radio (cms):"], Esfera, "esfera.png")

    def abrir_piramide(self):
        VentanaFigura(self, "Pirámide", ["Base (cms):", "Altura (cms):", "Apotema (cms):"],
                      Piramide, "piramide.png")
```

```
    def abrir_cubo(self):
        VentanaFigura(self, "Cubo", ["Lado (cms):"], Cubo, "cubo.png")

    def abrir_prisma(self):
        VentanaFigura(self, "Prisma", ["Largo (cms):", "Ancho (cms):", "Altura (cms):"],
                      Prisma, "prisma.png")
```

Archivo 9: principal_figuras.py

```
from ventana_figuras import VentanaFiguras


class Principal:
    def main():
        ventana = VentanaFiguras()
        ventana.mainloop()

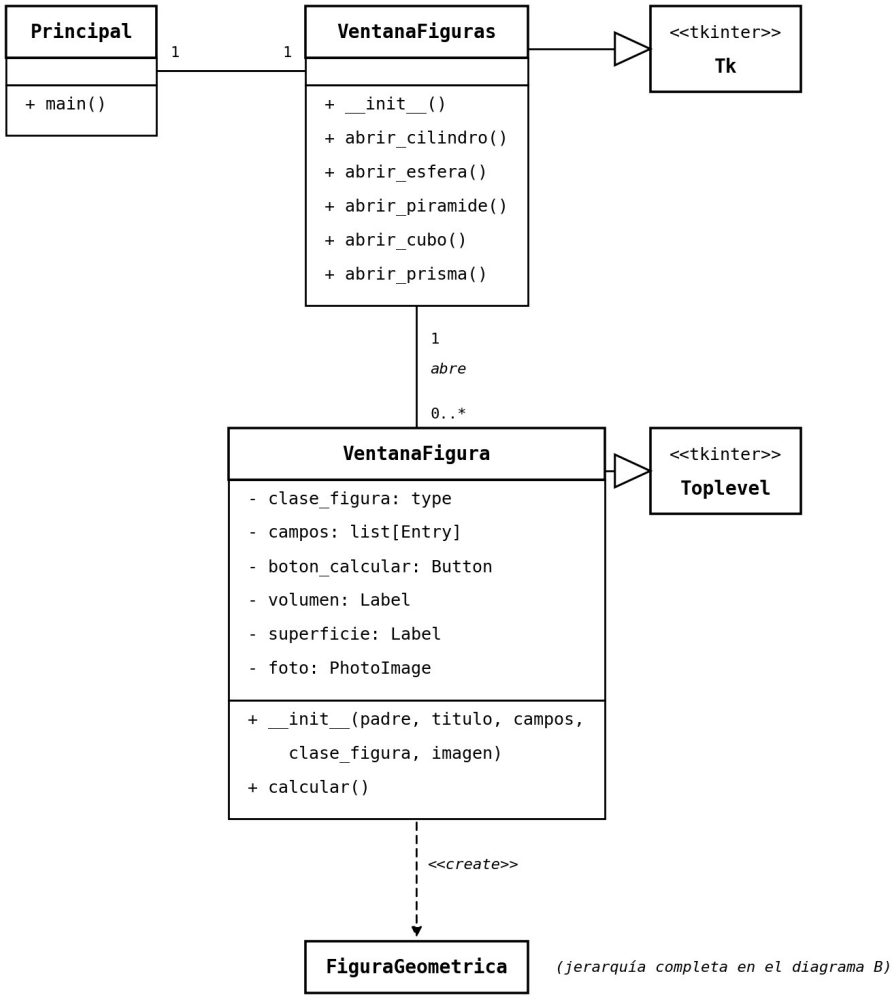

if __name__ == "__main__":
    Principal.main()
```

Interfaz de usuario:

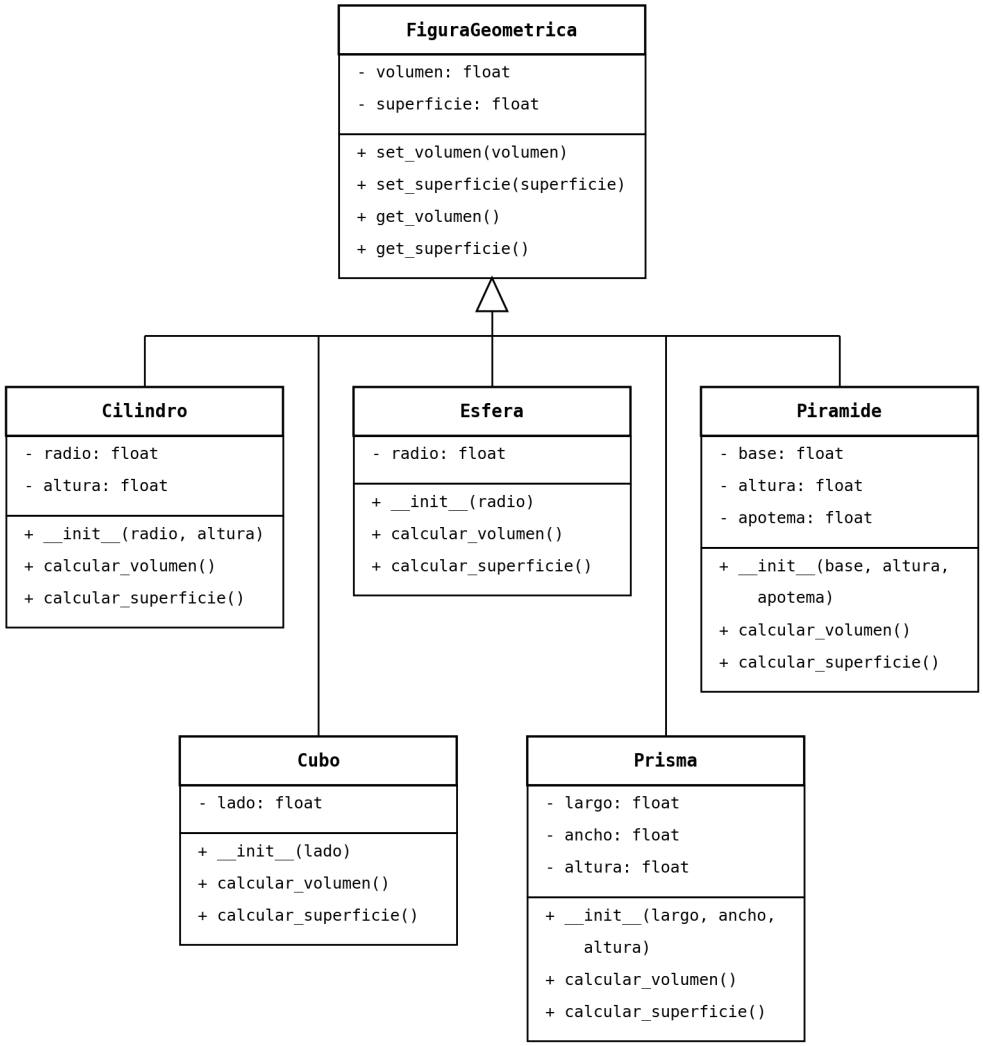
Captura de pantalla pendiente *a) Ventana principal*	*Captura de pantalla pendiente* *b) Cilindro*	*Captura de pantalla pendiente* *c) Esfera*
Captura de pantalla pendiente *d) Pirámide*	*Captura de pantalla pendiente* *e) Cubo*	*Captura de pantalla pendiente* *f) Prisma*

Datos de ejemplo: cilindro (radio 3, altura 10), esfera (radio 5), pirámide (base 6, altura 4, apotema 5), cubo (lado 4) y prisma (3 × 4 × 5).

Diagrama de Clase:

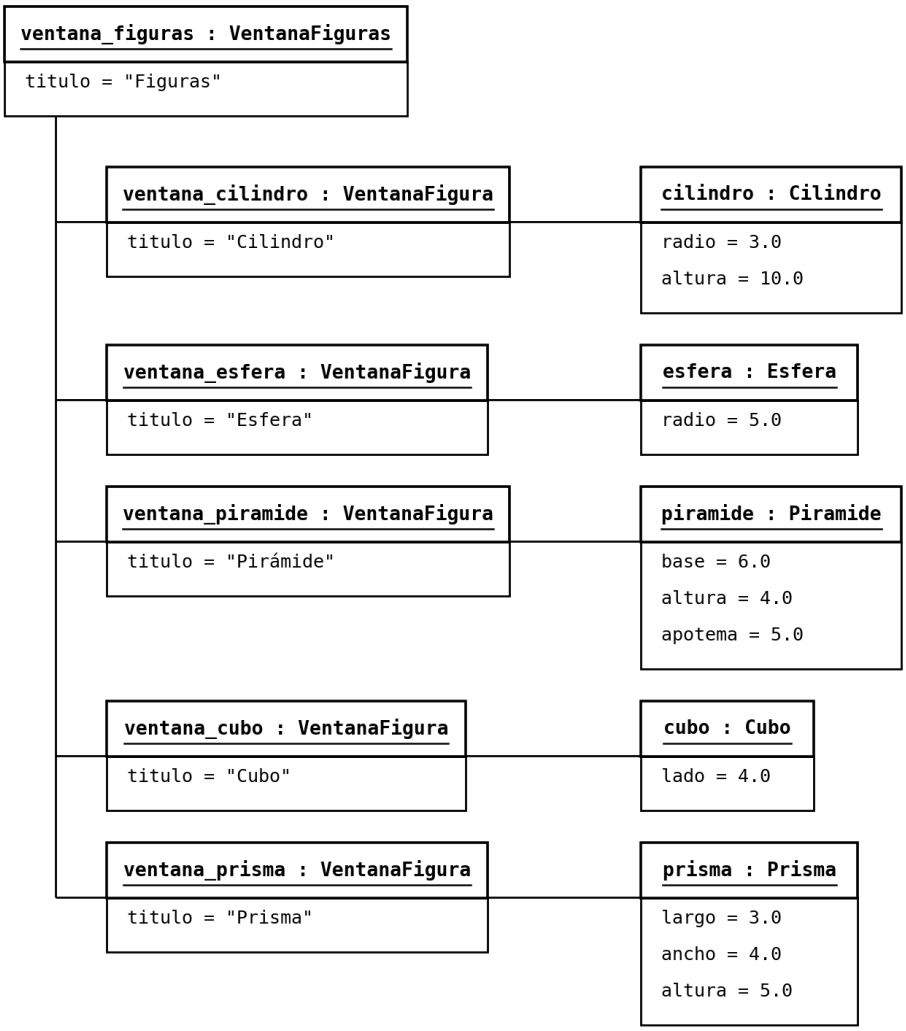


A) Interfaz gráfica

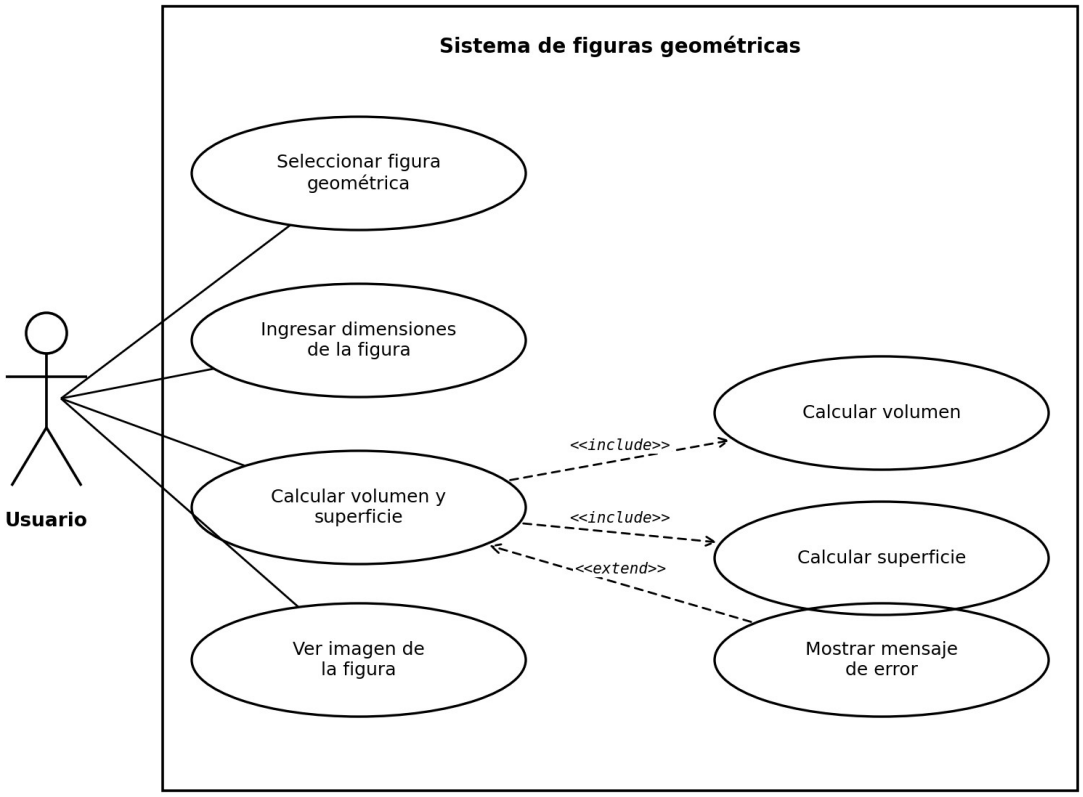


B) Modelo: jerarquía de figuras geométricas

Diagrama de Objetos:



Casos de uso:



Caso de uso	Descripción
Seleccionar figura geométrica	En la ventana principal el usuario pulsa el botón de la figura que quiere calcular (cilindro, esfera, pirámide, cubo o prisma) y se abre su ventana.
Ingresar dimensiones de la figura	El usuario escribe en los campos de texto las medidas de la figura en centímetros.
Calcular volumen y superficie	Al pulsar Calcular se crea la figura con los datos ingresados y se muestran su volumen y su superficie.
Calcular volumen / superficie	Operaciones propias de cada figura, incluidas en el caso de uso anterior.
Ver imagen de la figura	Parte propuesta: cada ventana muestra la imagen de su figura geométrica.
Mostrar mensaje de error	Se muestra cuando un campo está vacío, no es un número o es menor o igual a cero.

URL en GitHub:

[Pegar aquí el URL de GitHub: ejercicio 8.3]

URL del repositorio en GitHub:

[Pegar aquí el URL de GitHub: repositorio completo]